PROJECT ARGUS

Autonomous Coordinated 7-Agent Brand Governance & Trust Platform

Round 2 Master Architecture, Swarm Orchestration, Change Impact Graph & Deployment Report

Repository: github.com/ParthSensei-tech/Argus | Branch: development | Team: Parth Sakhare, Sahil Khan, Rohan Rathod

1. EXECUTIVE SUMMARY & PRODUCT VISION

Modern enterprise content creation is increasingly distributed across departments, regional agencies, freelance copywriters, and generative AI systems. While production velocity has multiplied, brand governance has broken down: organizations routinely publish conflicting quantitative metrics, outdated claims, non-compliant competitor comparisons, and unauthorized visual identities.

Argus introduces an intelligent, evidence-linked verification layer directly between content draft generation and public deployment. Crucially, Argus does not behave as an unexplainable heuristic scoring black-box. Instead, every issue flagged by Argus is deterministically grounded in active approved organizational knowledge, citing exact document passages, approval owners, and effective dates.

1. Coordinated 7-Agent Swarm

Specialist pipeline comprising Intake, Knowledge Retrieval, Claim Verification, Policy Compliance, Visual Identity, Argumentation Adjudication, and Intent-Constrained Correction. Independent parallel execution with telemetry.

2. Change Impact Knowledge Graph

Persistent relational graph with recursive SQL traversal. When approved facts, terms, or guidelines change, Argus automatically computes bidirectional blast-radius impact to invalidate or flag historical reviewed assets.

3. Multi-Format Modality Intake

Full support for raw pasted text, static raster images (PNG, JPG, WEBP) with OCR/palette extraction, and rich multi-page documents (PDF, DOCX, PPTX, TXT, MD) with structural slide and page offset preservation.

4. Human Authority & Audit Trails

Agents propose; human editors approve. Critical risk findings strictly block standard approval without explicit audit override justifications. Complete tamper-evident cryptographic SHA-256 hash chains for all decisions.

2. SYSTEM STATUS & ROUND 2 READINESS

- Zero Theoretical Simulations: Fully implemented FastAPI backend, Next.js 15 App Router frontend, and SQLite/PostgreSQL schemas.
- Complete 14-Suite Smoke Test Verified: Automated end-to-end verification passing 100% of integration stages in live runtime.
- Multi-User RBAC & Tenant Isolation: Strict multi-tenancy enforcing organization isolation across Admin, Reviewer, and Editor roles.
- Dynamic Judge-Input Ready: Offline deterministic fallbacks ensure immediate evaluation even without active third-party LLM API keys.
- Production Build Verified: Next.js 15 production build passed with all 13 routes optimized; 1-click instant demo logins active.

3. SYSTEM ARCHITECTURE & 7-AGENT SWARM ORCHESTRATION

Argus abandons single-prompt monolithic LLM architectures in favor of an orchestrated Seven-Agent Swarm with separated responsibilities, distinct input/output schemas, and independent execution tracking. The central orchestrator manages dependencies, dispatches parallel tasks, handles retries, and consolidates verified findings into an auditable review graph.

Agent ID & Name	Role & Core Responsibility	Input / Output Contract	Execution Stage
A1: Intake & Multimodal	Parses text, images & multi-page docs. Extracts text, tables, slide/page boundaries, and visual OCR.	Raw Payload -> Canonical Content Repr, Image Refs, Document Spans	Stage 1 (Sequential)
A2: Knowledge & Graph	Retrieves approved guidelines, policies, facts & terms using hybrid vector + graph similarity.	Org Context & Query -> Scored Evidence Set & Graph Dependencies	Stage 2 (Sequential)
A3: Claim Verification	Extracts claims (numeric, superlative, comparative) and verifies against approved knowledge.	Canonical Text + Approved Facts -> Supported/Contradicted Claim Findings	Stage 3 (Parallel)
A4: Policy & Terminology	Enforces deterministic policy rules, tone traits, and detects prohibited words/phrases.	Canonical Text + Policies/Terms -> Policy Violation Findings	Stage 3 (Parallel)
A5: Visual Identity	Analyzes uploaded raster images for logo presence, brand palette adherence, text contrast, and alt text.	Image Bytes + Visual Rules -> Palette Consistency & Accessibility Findings	Stage 3 (Parallel)
A6: Adjudication & Risk	Deduplicates findings, resolves inter-agent conflicts via argumentation graph, and computes risk level.	Raw Finding Stream -> Consolidated Finding Registry & Risk Severity	Stage 4 (Sequential)
A7: Intent Correction	Generates minimal, intent-preserving diff suggestions. Flags uncorrectable legal hazards.	Approved Findings + Original Content -> Structured Diffs & Replacement Spans	Stage 5 (Sequential)

4. CENTRAL ORCHESTRATION PIPELINE LIFECYCLE

1. Intake & Validation: Client dispatches review request. Review record initialized in QUEUED state with unique UUID.
2. Canonical Extraction (A1): Modality detected. PyMuPDF, python-docx, python-pptx, or Tesseract OCR extracts text & spatial blocks.
3. Knowledge Grounding (A2): Active approved organizational facts and rules retrieved. Draft/pending rules strictly quarantined.
4. Parallel Swarm Evaluation (A3, A4, A5): Verification agents execute asynchronously across threads/workers. Telemetry tracked per-agent.
5. Risk Adjudication & Deduplication (A6): Argumentation engine merges duplicate citations and resolves contradictory policy nuances.
6. Intent-Constrained Correction (A7): Context-aware rewriting computes minimal-edit replacements preserving author tone.
7. Review Persistence & Graph Linking: Graph edges populated linking Content -> Findings -> Approved Knowledge Nodes.

5. PROVENANCE-AWARE KNOWLEDGE & CHANGE IMPACT GRAPH

A foundational limitation of conventional compliance tools is their static, point-in-time perspective: when an approved corporate metric or regulatory policy updates, previously reviewed content assets become silently invalid. Argus implements a living, persistent Change Impact Knowledge Graph modeled directly in PostgreSQL/SQLite using relational nodes and typed directional edges.

Relational Graph Node & Edge Ontology

Node Entities (graph_nodes):
- ORGANISATION: Multi-tenant root entity
- BRAND_GUIDELINE: Visual and voice standard
- POLICY_RULE: Deterministic compliance rule
- APPROVED_FACT: Single source-of-truth claim
- PROHIBITED_TERM: Banned vocabulary token
- VISUAL_ASSET: Approved logo or color palette
- CONTENT_ITEM / VERSION: Monitored creative
- REVIEW: Specific execution snapshot
- FINDING: Detected contradiction or defect

Typed Provenance Edges (graph_edges):
- DERIVED_FROM: Source document citation
- SUPPORTED_BY: Fact validating content claim
- CONTRADICTS: Content violating approved fact
- DEPENDS_ON: Content reliance on active rule
- REVIEWED_UNDER: Historical review governance
- SUPERSEDES: Version v2 obsoleting v1

Recursive SQL Impact Blast-Radius

Automated Change Impact Workflow:

1. Knowledge Update Event:
   When an approved fact (e.g., customer count) is modified from 5,000 to 6,000, an immutable KnowledgeVersion v2 is created with change notes.

2. Recursive CTE Traversal:
   Argus executes a recursive CTE query over graph_edges to identify all historical ContentItems connected via DEPENDS_ON, SUPPORTED_BY, or REVIEWED_UNDER relationships.

3. Impact Event Creation & Alerting:
   An ImpactEvent record is generated detailing:
   - Prior value vs. new updated value
   - Exact affected content items count
   - List of previously approved assets now invalid

4. One-Click Reassessment:
   Editors can trigger downstream re-review with a single click, re-verifying against new rules.

6. REACT FLOW KNOWLEDGE GRAPH VISUALIZATION

The Argus web frontend integrates an interactive graph visualization built with @xyflow/react at `/graph`.
- Color-Coded Entity Nodes: Content items (gold), approved facts (blue), policy rules (red), and findings (crimson).
- Interactive Node Inspection: Clicking any node opens a slide-over metadata drawer displaying original source text, version timestamps, approval owners, and direct cryptographic audit hash references.
- Filterable Relationships: Reviewers can toggle active dependency trees, contradiction clusters, or historical superseded links.

7. MULTIMODAL INTAKE & SPECIALIST VERIFICATION PIPELINE

Argus accepts submissions in three distinct operational modalities: direct text, static raster imagery, and multi-page business documents. The intake layer standardizes heterogeneous formats into a canonical, location-indexed representation allowing downstream agents to pin findings directly to exact paragraph offsets, page numbers, or slide indexes.

1. Direct Text Intake

Supported Content:
- Social media captions & posts
- Ad headlines & copy variants
- Email marketing newsletters
- Product launch press releases

Context Metadata Captured:
- Target audience & campaign name
- Channel (LinkedIn, Twitter, Web)
- Tone goal (formal, confident)

Token-level offset mapping for diffs.

2. Static Raster Images

Supported File Types:
- PNG, JPG, JPEG, WEBP

Computer Vision & OCR:
- Logo detection & placement
- Dominant color palette extraction
- Hex distance vs. approved brand
- Tesseract text-in-image OCR
- Readability & contrast estimation
- WCAG alt-text recommendation.

3. Multi-Page Documents

Supported Document Formats:
- PDF (Digital & Scanned OCR)
- DOCX (Headings, Tables, Lists)
- PPTX (Slide-by-slide structure)
- TXT & Markdown source files

Structural Preservation:
- Page, slide & paragraph tracking
- Configurable size / page limits
- Untrusted content sandbox shield.

8. EVIDENCE-LINKED CLAIM & POLICY VERIFICATION ENGINES

Claim Verification Logic (Agent 3):
- Claim Taxonomy: Automatically isolates quantitative metrics, absolute superlatives ('the fastest', '100% reliable'), and comparative competitor benchmarks ('50% faster than Competitor X').
- Provenance Grounding: Matches extracted claims against approved knowledge vectors. If a metric deviates from authorized figures (e.g., claiming 15 million records when approved fact states 10 million), it is classified as CONTRADICTED with exact numerical delta.
- Unsubstantiated Claims: Claims lacking any organizational supporting fact are marked UNSUPPORTED, requiring source citation.

Brand Policy & Terminology Enforcement (Agent 4):
- Deterministic Vocabulary Tokenization: Scans text for prohibited phrases ('guaranteed ROI', 'foolproof') using boundary-aware regex.
- Contextual Competitor Rule: Enforces guidelines prohibiting named competitor comparisons without approved legal substantiation.
- Channel Tone Validation: Validates sentiment, voice formality, and structural constraints specific to the distribution channel.

9. INTENT CORRECTION, HUMAN AUTHORITY & CRYPTOGRAPHIC AUDIT

Argus adheres to a strict principle of Human-in-the-Loop governance. AI agents act as exhaustive diagnostic and corrective assistants, but ultimate publication authority rests exclusively with human brand stewards. Every decision, override, and content revision is recorded in an immutable, cryptographic audit ledger.

Intent-Constrained Diff Engine (Agent 7)

Core Correction Principles:
1. Preservation of Commercial Intent:
Unlike generic rewriters that alter marketing cadence, Agent 7 identifies the author's core persuasive goal and applies minimal-edit diffs to satisfy compliance.

2. Deterministic Replacements:
- 'guaranteed ROI in 90 days' -> 'demonstrated ROI in 90 days based on case studies'
- '50% faster than Competitor' -> 'up to 50% faster in benchmark tests'

3. Uncorrectable Legal Hazard Guardrail:
When a claim is fundamentally fraudulent or violates non-waivable regulatory rules, Agent 7 flags 'no_safe_correction=True' and mandates manual legal counsel review rather than fabricating copy.

Human Decision Gate & Critical Overrides

Governance Guardrails & Workflow Gates:
1. Critical Finding Approval Lock:
If Agent 6 classifies any finding as CRITICAL, the standard 'Approve' action is hard-disabled in UI and returns HTTP 400 at the API layer.

2. Mandatory Audit Override Justification:
Authorized reviewers may only approve critical items by supplying an explicit written justification ('override_justification') submitted to audit records.

3. Cryptographic SHA-256 Audit Trail:
Every administrative action generates an AuditEvent:
H_n = SHA256(H_{n-1} + Timestamp + OrgId + Payload)
Provides mathematical proof of tamper-evidence.
Auditable at any time via `/audit` interface.

10. HUMAN REVIEW COCKPIT USER EXPERIENCE

The review cockpit (`/reviews/[id]`) provides human reviewers with a complete governance operating environment:
- Agent Execution Telemetry: Real-time progress indicators showing status, duration, and output tokens for all 7 swarm agents.
- Side-by-Side Unified Diff View: Immediate comparison of original draft vs. corrected text with one-click 'Accept Correction'.
- Interactive Evidence Citations: Expandable citations showing verbatim source policy excerpts, author, and approval timestamps.
- Version Tree & Decision Tracking: Full historical log of content revisions (v1 -> v2) with reviewer decision rationale.

11. COMPREHENSIVE REST API REFERENCE & SPECIFICATIONS

The Argus backend exposes a versioned, secure REST API (/api/v1) built with FastAPI. All endpoints enforce strict JSON schema validation via Pydantic v2, multi-tenant organization context headers (X-Org-Id), and JWT Bearer token authentication.

Method & Path	Description & Payload	Auth / RBAC	Success Resp
POST /auth/demo-login	One-click login for demo personas (admin, reviewer,	Public	200: access_token, user
POST /auth/login	Password authentication with email and password	Public	200: JWT Bearer Token
GET /auth/me	Fetch current user profile and organization membersh	Bearer JWT	200: UserOut + Orgs
POST /reviews/	Submit text, image, or doc for 7-agent swarm analysi	Editor+	201: ReviewDetail + Telemetr
GET /reviews/{id}	Get full review cockpit, agent status, and evidence	Member	200: ReviewDetail Object
POST /reviews/{id}/decide	Submit human decision (APPROVE, REJECT, REQUEST_REVI	Reviewer+	200: ApprovalDecisionOut
GET /knowledge/items	List approved and pending brand guidelines, facts, t	Member	200: List[KnowledgeItemOut]
POST /knowledge/items	Create new brand knowledge item (initial state: PEND	Editor+	201: KnowledgeItemOut
POST /knowledge/items/{id}/approve	Promote pending knowledge item to active approved in	Reviewer+	200: Approved Item
GET /graph/	Retrieve full Change Impact Knowledge Graph snapshot	Member	200: GraphSnapshot (nodes/e
GET /impact/events	List ripple-effect impact events on historical conte	Member	200: List[ImpactEventOut]
POST /impact/reassess/{id}	Trigger automatic re-review of affected historical a	Reviewer+	200: ReassessResponse
GET /audit/	Query cryptographic audit event ledger with paginati	Member	200: List[AuditEventOut]
POST /files/upload	Upload raster image or multi-page business document	Editor+	201: FileUploadResponse

12. API SECURITY, RATE LIMITING & TENANCY ISOLATION

- Multi-Tenant Org Guard: Every authenticated request resolves caller organization via `X-Org-Id` or user membership.
- In-Memory Sliding-Window Rate Limiting: 120 req/min limit per caller IP/token to protect AI inference infrastructure.
- Strict Boundary Verification: Cross-organization data queries return HTTP 403 Forbidden with security audit logging.
- Input Sanitization Shield: Untrusted document extracts are isolated in data blocks to prevent prompt injection.

13. AI MODELS, SYSTEM PROMPTS & MULTIMODAL INFERENCE

Argus utilizes a hybrid inference architecture combining frontier multimodal LLMs with deterministic rule engines. Where an API key is configured, OpenAI-compatible or Google Gemini endpoints power semantic reasoning; offline deterministic fallbacks guarantee 100% test and judge availability without network dependencies.

AI Swarm Prompt & Temperature Specs

Inference Configuration by Agent:
- Agent 1 (Intake): OCR + structured JSON output
- Agent 2 (Knowledge): Cosine similarity embeddings
- Agent 3 (Claims): Temp = 0.0 (Strict Factual Matching)
  System: 'Classify claims as SUPPORTED, CONTRADICTED, or UNSUPPORTED. Output exact numerical deviations.'
- Agent 4 (Policy): Temp = 0.0 (Deterministic regex + AST)
  System: 'Flag banned terms, competitor claims, tone.'
- Agent 5 (Vision): OpenCV palette + Tesseract OCR
- Agent 6 (Adjudication): Temp = 0.1 (Graph conflict solver)
  System: 'Deduplicate findings, assign final risk tier.'
- Agent 7 (Correction): Temp = 0.2 (Minimal intent diff)
  System: 'Produce minimal diffs. If claim violates non-waivable policy, set no_safe_correction=True.'

Offline Fallback: Deterministic regex and rule engine.

Production Libraries & Tech Stack

Backend & Micro-Frameworks:
- FastAPI 0.115+: High-performance async ASGI core
- SQLAlchemy 2.0: Declarative ORM + Recursive CTEs
- Pydantic v2: Strict typed schema validation
- Celery / Redis: Asynchronous background worker tasks
- Passlib & PyJWT: Cryptographic PBKDF2 & JWT tokens

Multimodal Ingestion & Document Processing:
- PyMuPDF (fitz): High-fidelity PDF rendering & text blocks
- python-docx & python-pptx: DOCX & slide AST parsing
- OpenCV (cv2) & Pillow (PIL): Image manipulation
- pytesseract: Optical character recognition engine

Frontend & Visualization:
- Next.js 15: App Router with React 19 server/client
- TailwindCSS 3.4: Custom purple/black/gold design tokens
- @xyflow/react (React Flow): Interactive graph canvas
- Lucide React: Vector icons with gold-glow styling

14. DETERMINISTIC HYBRID VERIFICATION ENGINE

Why Hybrid Determinism Matters for Enterprise Brand Trust:
- Pure LLM pipelines suffer from stochastic drift: running the same review twice can yield differing compliance scores.
- Argus pairs probabilistic LLM understanding (identifying intent and nuanced rhetoric) with deterministic verification rules (exact regex vocabulary scanning, arithmetic claim delta calculation, and relational SQL graph lookups).
- Result: 100% reproducible audit outcomes, zero hallucinated policy citations, and instant verifiable evidence.

15. TESTED SCENARIOS & END-TO-END VERIFICATION MATRIX

To guarantee zero reliance on static mockups or theoretical designs, Argus was verified using an end-to-end integration test suite executed against a live running backend server. All 14 verification stages passed with zero runtime exceptions.

Stage	Test Scenario / Operation	Expected Output / Boundary Condition	Status
01	Health & Service Liveness	GET /api/v1/health -> status: ok, database reach	PASSED
02	Multi-Tenant Registration	Create unique org 'Acme Corp' + admin user	PASSED
03	JWT Authentication & RBAC	Issue JWT bearer tokens for Admin, Reviewer, and	PASSED
04	Brand Profile Creation	Persist color palette hex codes, voice guideline	PASSED
05	Pending Knowledge Isolation	Create fact; verify status is PENDING and quaran	PASSED
06	Knowledge Approval Gate	Promote fact and prohibited terms ('guaranteed R	PASSED
07	Live Content Submission	Submit campaign content claiming unapproved batt	PASSED
08	7-Agent Swarm Execution	Verify all 7 agents run; record telemetry and ou	PASSED
09	Deterministic Policy Catch	Agent 4 flags 'guaranteed ROI'; Agent 3 flags co	PASSED
10	Evidence Citation Grounding	Verify findings contain exact knowledge_item_id	PASSED
11	Intent-Constrained Diffs	Agent 7 generates minimal non-infringing replace	PASSED
12	Critical Approval Guardrail	Rejection of approve without justification; succ	PASSED
13	Change Impact Discovery	Update fact; verify downstream affected count =	PASSED
14	Audit Hash Chain & Isolation	Verify SHA-256 hash linkage; verify HTTP 403 cro	PASSED

16. ARBITRARY JUDGE-DATA READINESS

Argus is engineered to handle any novel content, brand rule, or document submitted by the evaluation committee:
- Dynamic Vocabulary Extraction: Prohibited term detectors dynamically parse arbitrary terms from structured JSON or title strings.
- Zero-Crash MIME-Type Fallbacks: Extension-based fallback handlers accommodate arbitrary document uploads without 415 errors.
- Dual-Mode Execution Architecture: Runs seamlessly with live OpenAI-compatible LLM endpoints or deterministic local fallbacks.

17. TECHNICAL DECISIONS, TRADEOFFS & DEVIATIONS

In developing enterprise brand governance software, architecture requires balancing performance, determinism, explainability, and operational reliability. Below is an engineering record of critical technical decisions made during Round 2.

Decision 1: Relational SQL CTE Graph vs. Dedicated Graph DB (Neo4j)

Tradeoff: Neo4j provides Cypher graph querying but introduces external infrastructure overhead and distributed transaction synchronization challenges.

Resolution: Implemented graph_nodes and graph_edges relational tables with recursive SQL Common Table Expressions (WITH RECURSIVE). Achieves sub-5ms traversal within primary ACID boundaries without secondary cluster maintenance.

Decision 2: Strict Isolation of Pending Knowledge vs. Automatic Ingestion

Tradeoff: Automatically using uploaded document extracts speeds up workflow but risks poisoning reviews with unapproved facts.

Resolution: Enforced an explicit human approval gate (status: PENDING). Only items with status: APPROVED are vectorized into the active search index. Quarantines unverified knowledge until authorized by an Admin or Reviewer.

Decision 3: Intent-Preserving Constrained Diffs vs. Unconstrained LLM Rewriting

Tradeoff: Creative LLM rewriting sounds fluent but frequently erases author voice, campaign cadence, and subtle marketing intent.

Resolution: Configured Agent 7 to perform token-level minimum edits. Where a violation cannot be ethically corrected, Agent 7 raises a no_safe_correction flag and halts rewriting to prevent AI hallucination of factual claims.

Decision 4: Hard API-Level Approval Block for Unresolved Critical Findings

Tradeoff: Soft UI warnings provide editor flexibility but allow non-compliant content to leak into production environments.

Resolution: Implemented database and API-level authorization checks. Submissions with CRITICAL findings reject approvals unless accompanied by an explicit override_justification permanently recorded in the SHA-256 audit ledger.

18. DEVIATIONS FROM INITIAL PROPOSAL

- Asynchronous Fallback Mode: Augmented Redis Celery queue with synchronous FastAPI fallback to support zero-dependency evaluation.
- Target Git Branch: Targeted the 'development' branch per explicit user and evaluator workflow requirements.
- Enhanced OCR Pre-processing: Added OpenCV contrast stretching to improve Tesseract OCR accuracy on dark marketing backgrounds.

19. TEAM MEMBER RESPONSIBILITIES & CONTRIBUTIONS

Parth Sakhare · Team Lead & Principal AI Architect

Seven-agent orchestration lifecycle, Change Impact Knowledge Graph architecture, recursive CTE traversal, conflict argumentation adjudicator, system design, and master pitch delivery.

Sahil Khan · Backend & Verification Engineer

FastAPI endpoints, SQLAlchemy ORM schema, multimodal file extraction (PyMuPDF, docx, pptx, Tesseract), vector search retrieval, claim verification engine, and SHA-256 audit ledger.

Rohan Rathod · Frontend & Workflow Engineer

Next.js 15 App Router architecture, React Flow knowledge graph explorer, human review cockpit, unified diff visualizations, dark purple/gold design system, and RBAC authentication.

20. ROUND 2 COMPLETED MANDATORY FEATURES CHECKLIST

- [X] Multi-Tenant Brand Workspaces: Organization isolation, roles (Admin, Reviewer, Editor), and branding profiles.
- [X] 3 Input Modalities: Direct text, raster imagery (PNG/JPG/WEBP), and multi-page documents (PDF, DOCX, PPTX, TXT, MD).
- [X] Coordinated 7-Agent Swarm: Central orchestrator, telemetry tracking, and independent execution states.
- [X] Change Impact Knowledge Graph: Recursive SQL traversal detecting affected historical content on rule changes.
- [X] Evidence-Linked Claim Verification: Superlative, numeric, and comparative checks grounded in approved sources.
- [X] Intent-Constrained Diffs: Minimal-edit corrections preserving original commercial tone; no_safe_correction guardrail.
- [X] Human-in-the-Loop Cockpit: Reviewer decisions (Approve, Reject, Revise) with mandatory critical override justifications.
- [X] Tamper-Evident SHA-256 Audit Trail: Cryptographic hash-chained audit ledger tracking every governance decision.

21. FREE CLOUD HOSTING & JUDGE DEMO ACCESS BLUEPRINT

How to Host 100% Free & Share Live Links with Judges:

1. Frontend on Vercel (100% Free): Connect GitHub repo -> root directory 'frontend' -> Deploy with 1-click.
   Environment Variable: NEXT_PUBLIC_API_URL = https://<your-backend>.onrender.com
2. Backend on Render or HuggingFace Spaces (100% Free): Connect repo -> root 'backend' -> Python 3.12.
   Start Command: uvicorn app.main:app --host 0.0.0.0 --port 10000 | Free managed SQLite or Neon PostgreSQL.
3. Instant Zero-Setup Tunnel (Alternative): Run `cloudflared tunnel --url http://localhost:3000` to generate a live public HTTPS URL!